DEMOSTRACIÓN DEL SISTEMA · VIDEOJUEGO NEXA LEGENDS

Generación de casos de prueba funcionales con LLM

Caso: compra de paquetes de monedas virtuales (REQ-001) en Nexa Games Studios S.A.C.

1 El caso de la demo

2 Cómo se relaciona la información

3 Tipos de escenario

4 Valores límite

5 El resultado

QUÉ DECIR

"Buenos días, profesora. Le presentamos nuestro sistema basado en modelos extensos de lenguaje para generar casos de prueba funcionales en la compra de monedas virtuales. Para la demostración usamos un caso sencillo del videojuego Nexa Legends."

1 · EL CASO DE LA DEMO

Se probó solo el camino feliz… y un error llegó a producción

5

reglas de negocio

RN-001 a RN-005 del requerimiento REQ-001

2

casos de prueba, ambos positivos

CP-001 (tarjeta, PEN) y CP-002 (PayPal, USD)

1

defecto en producción

DEF-002: un jugador recibió sus monedas dos veces

BÁSICO
500
monedas
18.90 PEN · 4.99 USD

INTERMEDIO
1 200
monedas
37.90 PEN · 9.99 USD

PREMIUM
2 500
monedas
74.90 PEN · 19.99 USD

PAGO
Tarjeta **PayPal**

MONEDA
PEN **USD**

REQ-001 · Compra de paquete de monedas. El jugador compra un paquete (Básico 500, Intermedio 1 200 o Premium 2 500 monedas), paga en **PEN o USD** con **tarjeta o PayPal**, y las monedas se acreditan cuando el pago es aprobado.

QUÉ DECIR

"El requerimiento REQ-001 permite comprar paquetes de monedas. Tiene cinco reglas de negocio, pero el equipo QA lo probó con solo dos casos, ambos del camino feliz. Por eso un error llegó a producción: un jugador recibió sus monedas dos veces. Ese es el problema de nuestra tesis: la cobertura de casos de prueba es insuficiente."

De lo que pide el negocio a lo que falló

REQ

Requerimiento

¿Qué necesita el negocio?

REQ-001 Comprar paquetes de monedas con tarjeta o PayPal.

→

HU

Historia de usuario

¿Cómo lo vive el jugador?

HU-001 "Como jugador quiero comprar un paquete para usar las monedas."

→

RN

Regla de negocio

¿Qué condición siempre se cumple?

RN-004 Si la confirmación llega dos veces, se acredita una sola vez.

→

CP

Caso de prueba

¿Cómo lo verificamos?

CP-001 Comprar el Básico con tarjeta en PEN → saldo 500.

→

DEF

Defecto

¿Qué falló?

DEF-002 Las monedas se acreditaron dos veces.

! **El hueco:** la regla RN-004 no tenía ningún caso de prueba. Por eso DEF-002 no se detectó en pruebas, sino en producción. Esta cadena completa es el **contexto que el sistema entrega al LLM**.

QUÉ DECIR

"La información histórica forma una cadena. El requerimiento dice qué necesita el negocio; la historia de usuario lo cuenta desde el jugador; las reglas de negocio son las condiciones que siempre deben cumplirse; los casos de prueba las verifican; y los defectos son las fallas encontradas. Aquí la regla RN-004 no tenía ninguna prueba, y por eso apareció el defecto DEF-002. Esta cadena es la que el sistema le entrega al LLM como contexto."

3 · TIPOS DE ESCENARIO

Cada caso de prueba responde a una situación distinta

Positivo

¿Funciona cuando todo sale bien?

EJEMPLO

Comprar el Básico con una tarjeta válida → recibe 500 monedas.

Lo que ya se probaba

Negativo

¿Rechaza lo que no debe aceptar?

EJEMPLO

Tarjeta rechazada → no recibe monedas. Sexta compra del día → se bloquea.

Faltaba

Alternativo

¿Funciona si el usuario toma otro camino?

EJEMPLO

El jugador cancela en el resumen o cierra PayPal sin pagar.

Faltaba

Excepción

¿Resiste fallas externas?

EJEMPLO

La pasarela envía dos veces la confirmación → se acredita una sola vez.

Faltaba · causó DEF-002

QUÉ DECIR

"Cada caso de prueba tiene un tipo de escenario. Positivo: todo sale bien. Negativo: el sistema debe rechazar algo inválido, como una tarjeta rechazada. Alternativo: el usuario toma otro camino válido, como cancelar. Excepción: falla algo externo, como una confirmación duplicada de la pasarela. El proceso manual se quedó en los positivos; los otros tres son los que se omiten, y ahí aparecen los defectos."

Los errores se esconden en los bordes de una regla

RN-005 · Máximo 5 compras aprobadas por día



Qué detecta

Si el programador escribió `compras < 5` en lugar de `compras <= 5`, la 5.ª compra se bloquearía por error. Solo probando la 5.ª y la 6.ª se descubre.

Por qué ahorra pruebas

No hace falta probar las compras 1, 2 y 3: basta con los valores alrededor del borde, que es donde se equivoca el código.

QUÉ DECIR

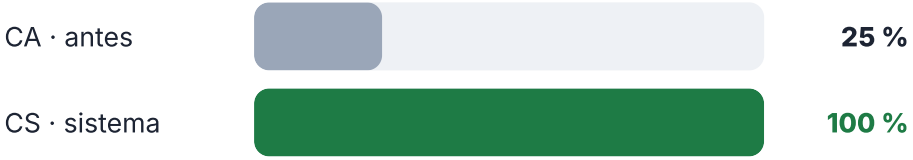
"Una de las técnicas que aplica el LLM es el análisis de valores límite. Los errores suelen estar en los bordes de una regla. Si la regla dice máximo cinco compras al día, probamos la quinta, que debe permitirse, y la sexta, que debe bloquearse. Así detectamos si el programador escribió 'menor que' en lugar de 'menor o igual que'."

5 · EL RESULTADO · INDICADOR IE4

De 1 a 4 tipos de escenario: +300 % de cobertura

Tipos de escenario cubiertos en REQ-001

	Positivo	Negativo	Alternativo	Excepción
Antes (manual)	✔	✘	✘	✘
Con el sistema	✔	✔	✔	✔



Reglas de negocio cubiertas: de 3 de 5 (60 %) a 5 de 5 (100 %). Ahora se prueba RN-004, la regla que causó DEF-002.

Cálculo de la mejora de cobertura

```
MC = (CS − CA) / CA × 100
MC = (100 − 25) / 25 × 100
MC = 75 / 25 × 100
MC = 300 %
```

Meta IE4: MC ≥ 20 % ✓



Las cifras del sistema corresponden a los casos generados en la demo; deben confirmarse con el resultado real de cada generación.

QUÉ DECIR

"Comparamos el mismo requerimiento antes y después. Antes, los casos cubrían solo uno de los cuatro tipos de escenario: 25 %. Con el sistema, los casos generados cubren los cuatro: 100 %. Con la fórmula de nuestro indicador IE4, la mejora es cien menos veinticinco, entre veinticinco, por cien: 300 %, cuando la meta era 20 %. Además, ahora se prueba la regla RN-004, la que causó el defecto en producción."

GUION DE EXPOSICIÓN · ~5 MINUTOS

Textos a exponer, en orden

1 **Presentación** · 20 s
"Buenos días, profesora. Le presentamos nuestro sistema basado en LLM para generar casos de prueba funcionales en la compra de monedas virtuales, con un caso del videojuego Nexa Legends."

2 **El problema** · 40 s · pantalla: Ver trazabilidad de REQ-001
"REQ-001 tiene cinco reglas, pero se probó con solo dos casos positivos. Por eso un error llegó a producción: un jugador recibió sus monedas dos veces."

3 **La relación** · 40 s
"Requerimiento, historia, reglas, casos y defectos forman una cadena. RN-004 no tenía pruebas y por eso apareció DEF-002. Esta cadena es el contexto que recibe el LLM."

4 **Generación** · 60 s · pantalla: Generar casos
"Elegimos REQ-001, el sistema recupera la información relacionada, marcamos los cuatro tipos de escenario y las técnicas, y generamos. El analista revisa los casos antes de usarlos."

5 **Tipos de escenario** · 40 s · pantalla: casos generados
"Ahora tenemos positivos, negativos como la tarjeta rechazada, alternativos como cancelar la compra y de excepción como la confirmación duplicada, que es justo el defecto DEF-002."

6 **Valores límite** · 30 s
"Con valores límite, el sistema prueba la quinta compra, que se permite, y la sexta, que se bloquea: ahí es donde suele equivocarse el código."

7 **Resultado** · 40 s
"La cobertura de tipos de escenario pasó de 25 % a 100 %: una mejora de 300 %, frente a la meta de 20 % del indicador IE4."

8 **Cierre** · 20 s
"El sistema no reemplaza al analista QA: le entrega casos más completos a partir de la información histórica, y el analista y el líder QA los revisan antes de usarlos. Muchas gracias."